

Form Builder & Milestone Management — Working Session Log

Israeli Tax Authority HR — Full-Stack PoC • .NET 8 Web API (Clean Architecture) • Angular 22 (RTL / Hebrew) • EF Core 8 • SQL Server LocalDB
Generated 2026-09-03

About this document. This is a reconstruction of the working session — each request and the work carried out in response — assembled from the assistant's record of the session. It is not a byte-for-byte chat transcript; long tool outputs are summarised. The authoritative record of the code itself is the repository and the per-step notes under docs/ .

1. Overview

The session took the project from an empty folder to a working full-stack proof of concept, following a 12-step plan (ARCHITECTURE.md). Work proceeded in approved batches; no git operations were performed by the assistant — the user maintains the repository (github.com/YehuditJacobson/FormBuilder) and commits each step after review.

What was delivered

Area	Result
Architecture blueprint	Published design artifact + ARCHITECTURE.md (12-step plan, domain model, API contract, cloud notes)
Screen designs	5-artboard hi-fi mockup canvas, Hebrew / RTL, "clean enterprise" direction
Backend	Domain, Application (CQRS/MediatR), Infrastructure (EF Core), API (3 endpoints) — 55 tests
Frontend	Angular 22 standalone, RTL Hebrew, form builder + list + view, signal store — 23 tests
Database	SQL Server LocalDB (FormBuilder), migration applied, seeded
Verification	Full create → list → view round-trip confirmed end-to-end through the Angular dev proxy

2. Chronological log

Request 1 — *Design a full architecture for a form-builder application as described in the PDF, divided into steps. Server: .NET Core. Client: Angular. DB: SQL Server. Clean code, best practices, design patterns, as a senior developer.*

Analysed the exam brief (organizational form templates + dynamic approval routes for HR). Produced a design artifact "Form Builder Blueprint" and a full architecture: Clean Architecture backend (Domain / Application / Infrastructure / Api), CQRS with MediatR, Repository + Unit of Work, a normalized relational model (FormTemplate aggregate root owning FormField and ApprovalStep), a 3-endpoint API contract, an Angular Reactive-Forms client with a signal store, decision points (normalized vs. blob storage, template vs. instance), and Part 3 cloud / theory talking points. Delivered as a 12-step build sequence.

Request 2 — *Save this design in a doc in the folder and go ahead with step 0. Don't push to git. I want to approve the code. Write clean and clear as senior development.*

Saved `ARCHITECTURE.md` . Completed **Step 00 — solution scaffolding**:

- Solution + 4 source projects + 3 test projects, wired by the inward dependency rule.
- `Directory.Build.props` (net8.0, nullable, warnings-as-errors, analyzers, XML docs), `Directory.Packages.props` (Central Package Management), `tests/Directory.Build.props` .
- `.editorconfig` , `.gitignore` , `global.json` (SDK 8 pinned).
- Removed template boilerplate (`WeatherForecast` , `Class1` , `UnitTest1`).

Build: 0 warnings, 0 errors.

Request 3 — *Save the summary of what was done in a doc in the folder.*

Created `docs/step-00-scaffolding.md` . This per-step doc pattern continued for every subsequent step.

Request 4 — *Do 2 steps in parallel. Don't push. I want to approve all the code.*

Confirmed the pair: **Step 01 (Domain)** + **Step 06 (Angular scaffold)** — two independent tracks.

- **Step 01:** `Entity` `base`, `DomainException` , `DomainGuard` ; enums `FieldType` / `ApprovalActionType` / `TemplateStatus` ; `FormField` , `ApprovalStep` (internal ctors), `FormTemplate` aggregate root with encapsulated collections and invariant-enforcing methods (`AddField` , `RemoveField` , `ReorderFields` , approval-step equivalents, `Publish`). **31 unit tests**.
- **Step 06:** `npx @angular/cli@latest` → Angular 22, standalone, strict, SCSS, Vitest. Added `environments/` , `proxy.conf.json` (→ API on `:5052`), `strictTemplates` , HTTP client, lazy route shell. Replaced the CLI welcome page. **Build clean, 2 tests**.

Request 5 — *Before building the client components, provide the screens you will develop with clean UI/UX design — after I approve, we develop.*

Clarified: static hi-fi mockups vs. clickable prototype, and visual direction. User chose **static hi-fi mockups** + **"clean enterprise"** direction. Produced a 5-artboard `design canvas` : Form list (`GET /forms`), Create form (`POST /forms` — the core screen, with add-field buttons, dynamic field rows, approval-route rows, live preview), Saved-form view (`GET /forms/:id`), Validation states, Empty state. Public Sans, a single calm blue accent, soft borders, no drag-and-drop (reorder arrows).

Request 6 — *Looks amazing, but the site language needs to be Hebrew. Fix it and go ahead with the next 2 steps in parallel.*

Rebuilt all 5 artboards in **Hebrew, right-to-left** (`dir="rtl"` , Assistant webfont, logical CSS properties, mirrored directional icons) and republished the canvas to the same link. Recorded the Hebrew/RTL requirement for all subsequent client work.

Then **Step 02 (Application)** + **Step 03 (Infrastructure)**:

- **Step 02:** `Result` / `Result<T>` + `Error` ; abstractions (`IFormTemplateRepository` , `IFormTemplateQueries` , `IUnitOfWork` , `IDateTimeProvider` , `ICurrentUser`); DTOs; `ValidationBehavior` pipeline;

FormTemplateFactory ; three use cases (CreateFormTemplateCommand + validator, GetFormTemplatesQuery , GetFormTemplateByIdQuery); AddApplication() . **14 tests.**

- **Step 03:** ApplicationDbContext + one IEntityTypeConfiguration per entity (enum-as-string, cascade FKs, unique index on (FormTemplateId, Order) , collections mapped to private backing fields); FormTemplateRepository , FormTemplateQueries (SQL-side projections), UnitOfWork ; AddInfrastructure(config) with a provider switch (SqlServer / Sqlite / InMemory); InitialCreate migration (SQL Server schema); dotnet-ef local tool pinned. **5 persistence tests (SQLite in-memory).**

Request 7 — Approve the completed steps. Go to the next parallel pair.

Step 04 (API) + Step 07 (Angular core & shared):

- **Step 04:** Program.cs composition root (Serilog, JsonSerializerConverter , API versioning, Swagger, ProblemDetails + GlobalExceptionHandler , CORS, provider-aware DB init); FormsController (3 endpoints); ResultExtensions mapping; HttpContextCurrentUser . Verified live: 201 / 200 / 200 / 400 (ValidationProblemDetails) / 404 .
- **Step 07:** RTL index.html (lang="he" dir="rtl" , title כוונה טפסים); token + primitive stylesheets; TS models; FormTemplateApiService ; ApiError + error interceptor (parses RFC 7807, Hebrew fallbacks); shared <app-icon> (17 inline SVG icons). **7 client tests.**

During this step the assistant noticed a populated .git with the user's own remote and commits — confirmed the user manages git; the assistant continued to run no git commands.

Request 8 — Go ahead with the next parallel pair.

Step 05 (backend integration tests + seed) + Step 08 (form-builder page):

- **Step 05:** FormBuilderApiFactory (WebApplicationFactory<Program> , per-factory in-memory DB), FormsEndpointsTests (5 real HTTP round-trips). Fixed a bug where UseInMemoryDatabase(\$"...{Guid.NewGuid()}") created a fresh empty DB per request. Added DataSeeder (one sample template in Development). **5 tests → 55 backend total.**
- **Step 08:** typed BuilderForm with two FormArray S; minLengthArray validator; request mapper; Hebrew labels; <app-field-row> , <app-approval-step-row> , <app-form-preview> (live read-only render); FormBuilderPage — add / remove / reorder rows, form-level validation summary, Save disabled while invalid, submit → POST . **15 client tests.**

Request 9 — Go ahead with the next pair of steps in parallel.

Step 09 (list + view pages) + Step 10 (integration & polish):

- **Step 09:** FormListStore signal facade (templates / loading / error / isEmpty); FormListPage (loading / error / empty / table); FormViewPage (routed signal input, read-only render, 404 handling); refactored FormPreviewComponent to shared normalized inputs; routes wired (' / new / :id); builder now navigates to the new template on save. **23 client tests.**
- **Step 10:** ran the API + ng serve together; verified through the dev proxy: seeded template listed → POST new → list shows 2 (newest first) → GET /:id full ordered graph → 404 for unknown; CORS preflight 204 with the allow-origin header; SPA shell served as <html lang="he" dir="rtl"> . Removed dead code; README run section.

Request 10 — *How do I run the client from VS Code, serve from VS, and where / how do I see the database?*

Explained the workflow (API from Visual Studio on the `http` profile / port 5052; Angular from VS Code via `npm start` or the "ng serve" F5 launch; the proxy needs no CORS in dev) and how to view each provider's data (SQLite file, InMemory, SQL Server Object Explorer). Saved `docs/running-locally.md`.

Request 11 — *(PowerShell error) npm.ps1 cannot be loaded because running scripts is disabled on this system.*

Diagnosed the PowerShell execution policy. Fix: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` (or use `npm.cmd` / Command Prompt). Also noted `npm start` must run from `client/`, not the repo root. Added a Troubleshooting section to the doc.

Request 12 — *Can you run the client for me?*

Started the API (Development, so the seeder runs) and the Angular dev server. Client came up at `http://localhost:4200`; verified the `/api` proxy reached the API and returned the seeded template.

Request 13 — *I want to see the DB tables, but in Visual Studio I see the old DB, not the Form Builder one.*

Diagnosed: the API was running on **SQLite** (`src/FormBuilder.Api/formbuilder.db`), so nothing had ever been written to SQL Server; the database visible in Visual Studio (`FieldEvents` in `(localdb)\MSSQLLocalDB`) belongs to a different project. Offered a choice; user chose to switch to SQL Server / LocalDB.

Changed `appsettings.json` to `Provider: "SqlServer"` with `Server=(localdb)\MSSQLLocalDB;Database=FormBuilder;...`, restarted the API. Verified with `sqlcmd`:

```
FormBuilder database created in (localdb)\MSSQLLocalDB
Tables: __EFMigrationsHistory, FormTemplates, FormFields, ApprovalSteps
Rows:   FormTemplates 1 | FormFields 3 | ApprovalSteps 2
Seeded: "בקשת חופשה" (Published, CreatedBy = system)
```

Updated `docs/running-locally.md` and `README.md`.

Request 14 — *Run the client again.*

Client and API were still up; restarted the dev server cleanly. Verified the proxy still reaches the SQL Server-backed API.

Request 15 — *Save all this conversation to a PDF file in the folder.*

This document.

3. Final state

Test totals

Project	Tests
FormBuilder.Domain.Tests	31
FormBuilder.Application.Tests	14
FormBuilder.Infrastructure.Tests	5
FormBuilder.Api.IntegrationTests	5
Backend total	55
client (Vitest)	23

Solution layout

```
testApplication/
  ARCHITECTURE.md          full design + 12-step plan
  README.md                build / run / progress
  global.json Directory.Build.props Directory.Packages.props .editorconfig
  .config/dotnet-tools.json (dotnet-ef pinned)
  docs/                    step-00 ... step-10, running-locally.md, session-log.pdf
  src/
    FormBuilder.Domain      entities, enums, invariants (no dependencies)
    FormBuilder.Application  CQRS use cases, DTOs, validators, abstractions
    FormBuilder.Infrastructure EF Core: DbContext, configs, repos, queries, migration
    FormBuilder.Api          controllers, middleware, composition root
  tests/
    FormBuilder.Domain.Tests
    FormBuilder.Application.Tests
    FormBuilder.Infrastructure.Tests
    FormBuilder.Api.IntegrationTests
  client/                    Angular 22, RTL Hebrew
    src/app/core/            models, api service, error interceptor
    src/app/shared/ui/icon/
    src/app/features/form-builder/
      pages/ form-list.page form-builder.page form-view.page
      components/ field-row approval-step-row form-preview
      model/ builder-form (+ validators, mapper), labels
      state/ form-list.store
```

API contract

Method	Route	Success	Errors
POST	/api/v1/forms	201 + Location + { id }	400 ValidationProblemDetails
GET	/api/v1/forms	200 + summary[] (newest first)	—
GET	/api/v1/forms/{id}	200 + full detail	404 ProblemDetails

Running services at end of session

- API — `http://localhost:5052` (Swagger at `/swagger`), provider **SqlServer** / `(localdb)\MSSQLLocalDB`, database `FormBuilder`.
- Client — `http://localhost:4200` (`ng serve`, watch mode), `/api` proxied to the API.

Standing constraints observed throughout

- No git operations performed by the assistant; the user commits each approved step.
- Every step (or batch of steps) stopped for the user's review before continuing.
- A short summary doc written under `docs/` for each step.
- Client UI in Hebrew, right-to-left, "clean enterprise" visual direction.

Remaining

Step 11 — the Part 3 theory write-up (drafted in `ARCHITECTURE.md` §10: architecture rationale, normalized vs. blob storage, template vs. running instance, Azure deployment, security) and the screen recording covering architecture, the DB model, and cloud deployment.